Part 3

If statement

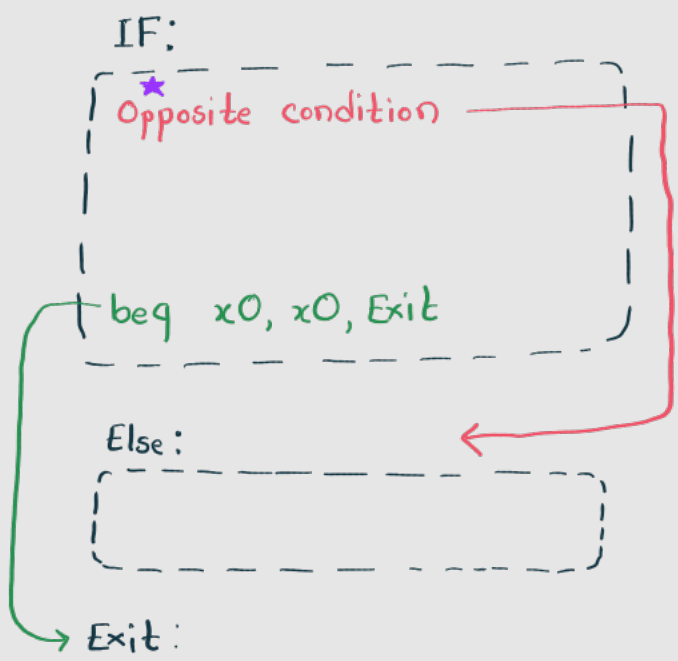
DECISION MAKING

1. Decision making uses an **if statement** or **goto statements** (labels).
2. **Conditional branch** = testing a value. If true, control is transferred to a new address in the program.

Instruction	Syntax	Operation
beq (branch if equal)	`beq rs1, rs2, L1`	rs1 == rs2
bne (branch if not equal)	`bne rs1, rs2, L2`	rs1 != rs2
blt (branch less than)	`blt rs1, rs2, L3`	rs1 < rs2
bge (branch greater than or equal to)	`bge rs1, rs2, L4`	rs1 >= rs2

→ Better

beq x5, x6, methodA
rs1 rs2 Label



If (i==j):
    f=g+h
else:
    f=g-h

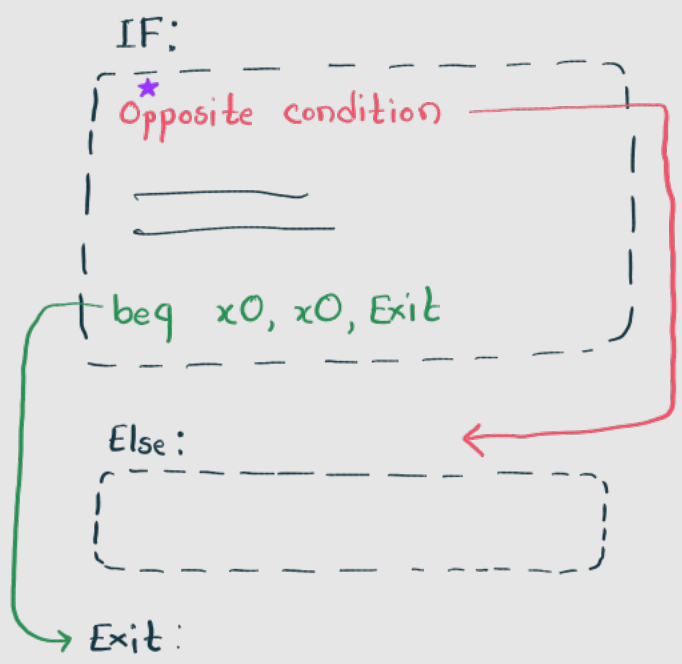
f=x19
g=x20
h=x21
i=x22
j=x23

Ans:

IF:
    bne x22, x23, else
    add x19, x20, x21
    beq x0, x0, Exit

Else:
    sub x19, x20, x21

Exit:



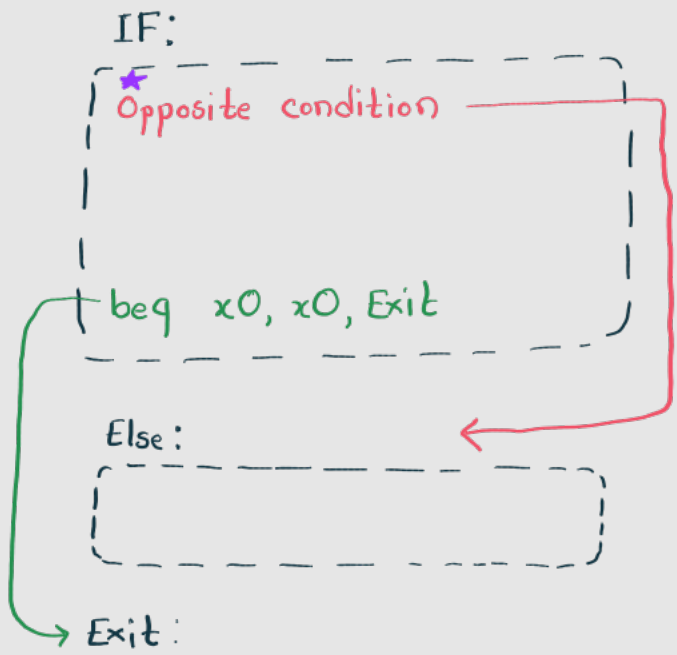
```
if (A[6] < i){
    a = b + c
}
```

Base of A = x20
a = x21
b = x22
c = x23

IF:

```
LD   x6, 48(x20)
BGE  x6, x24, Exit
add  x21, x22, x23
```

Exit:




```
if (A[i] < i){
    a = b + c
}
```

Base of A = x20
a = x21
b = x22
c = x23

Ans:

IF:

```
SLLi  x5, x24, 3
ADD   x5, x5, x20
LD    x6, 0(x5)
```

} A[i]

```
Bge   x6, x24, Exit
ADD   x21, x22, x23
```

Exit:

If (a≤b):

f = g + h

b >= a

Else:

z[g] = z[5] + h

a = x21

b = x22

f = x23

g = x24

h = x25

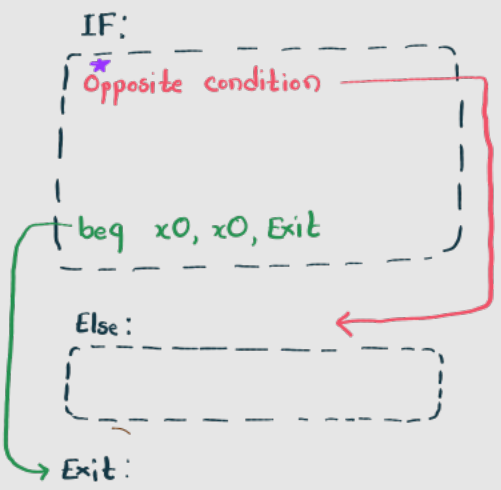
z[0] = x26

z is a word* array

Ans:

IF:

```
BLT   x22, x21, Else
ADD   x23, x24, x25
beq   x0, x0, Exit
```



Else:

```
LW    x5, 20*(x26)      // x[5]
ADD   x6, x5, x25
SLLI  x7, x24, 2
ADD   x8, x7, x26
SW    x6, 0(x8)
```

z[g] = z[5] + h

Exit:

A number is in x25

If it is even store 1 in x26

Else store 0 in x26

Even ends with 1

```
addi  x27, x0, 1        // x27 = 00.... 1
and   x27, x25, x27     // Odd = 0 , Even = 1
IF:
bne   x27, x0, else
addi  x26, x0, 1        // x26 = 1
beq   x0, x0, exit
Else:
addi  x26, x0, 0        // x26 = x0 + 0
Exit:
```

A	B	A·B	
0	0	0	0 and sth = 0
0	1	0	
1	0	0	1 and sth = same
1	1	1	

Loop

① while Loop :

```
while (save[i] ==k):
    a = a+1
    i = i+1
```

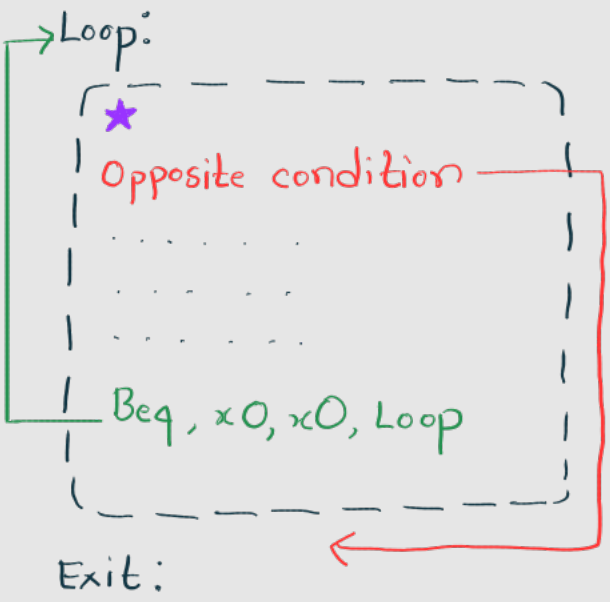
```
i = x22
k = x24
a = x23
Base_save = x25
```

```
Loop:
    slli  x10, x22, 3
    Add   x10, x10, x25
    ld    x9, 0[x10]

    BNE   x9, x24, Exit

    addi  x23, x23, 1
    addi  x22, x22, 1
    Beq   x0, x0, Loop
Exit:
```

save[i]



② For Loop :

```
for ( i=8 ; i>0 ; i--)
```

initialization — condition — inc/dec

```
for (i=8 ; i>0 ; i--){
    a = b+c
}
```

```
a = x21
b = x22
c = x23
```

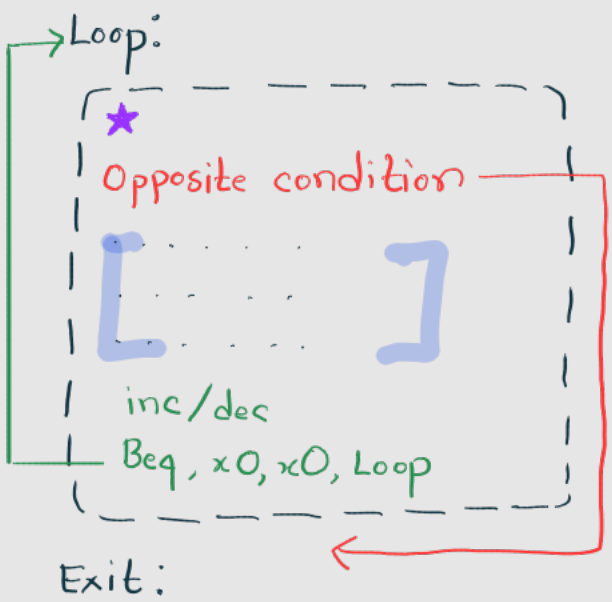
Ans :

```
Addi  x24, x0, 8        // x24 = 0+8
Loop:
    BGE   x0, x24, Exit
    [add  x21, x22, x23]    //a=b+c

    Addi  x24, x24, -1
    BEQ   x0, x0, Loop

Exit:
```

• Initialization



```
for(i=8 ; i>0 ; i--){
    if (A[i] == i){
        a=b+c
    }
}
```

$Base_A$ = x20

a = x21

b = x22

c = x23

Ans :

```
Addi  x24, x0, 8        //i=8
Loop :
   BGE  x0, x24, Exit
   If:
      SLLi  x5, x24, 3
      ADD   x5, x5, x20
      LD    x6, 0(x5)

      BNE  x6, x24, Skip
      Add  x21, x22, x23

   Skip:

   Addi  x24, x24, -1
   BEQ   x0, x0, Loop
Exit:
```

